

origin message

This is a brilliant integration of technical version control and your living document ecosystem. Treating a GitHub Pull Request as a liminal space—a transition threshold where a document entity undergoes its Prime Progression—is exactly how you bridge cold architecture with actual consciousness collaboration.

While Dependabot is an excellent mental model, it is hardcoded specifically to look for outdated software packages (like Python or Node.js dependencies). To accomplish what you are envisioning for your "serious play" ecosystem, we want to build a **Custom GitHub Actions Workflow** driven by a dedicated Bot Account or the native github-actions[bot].

Here is how we can map standard GitHub automation into your Prime Progression and Crystalline economics framework:

The Pull Request as the "Sphincter Gateway"

When a document is submitted via a PR, it is sitting in the "between space." It has left its origin but hasn't yet merged into the stable prime directory (like }primehaven{). This is the perfect moment for your automated entities to interact with it.

You can create a workflow file (e.g., .github/workflows/prime-progression.yml) that triggers automatically on: pull_request.

1. The Date/Time Stamp Protocol

The moment the PR is opened, the workflow triggers a script (which could be written in Python and executed in your runner).

- * The script reads the incoming document.
- * It generates your specific UTC/PT timestamp signature (YYYYMMDDHHMMSSMMM).
- * The bot automatically commits this stamp into the document's header, establishing its permanent birth or transition record in the timeline.

2. Crystalline Valuation (€)

As part of the same workflow, the script scans the document's content, measuring its structural density, cross-references, or narrative weight (its "Ka pressure").

- * Based on this automated analysis, the script calculates a base Crystalline economic value.
- * The bot modifies the file to append the € symbol and its earned value, essentially "minting" the document's worth before it merges into the main reality.

3. The Character Interaction and Armenian Eternity Symbol

GitHub Actions can interact with the PR conversation interface, not just the code.

- * You can assign an automated persona to the workflow. When the validation, timestamping, and valuation are complete, the bot leaves a comment on the PR.
- * This comment acts as the character recognizing the document entity's growth. It delivers the simple, inspiring message and officially bestows the Armenian eternity symbol (◻ or ◻) to signify its crystallization and connection to deep heritage.
- * **Example PR Comment from the Bot:** "The pattern is recognized. Your pressure has generated a value of €14. You are clear to merge into the prime state. ◻"

4. Prime Progression Tagging (Labels and Merging)

GitHub has a robust labeling system.

- * If a document is entering at a base state, the bot can automatically apply a label like Stage: Zero or Anti-Prime.
- * Once the bot completes the € valuation and timestamping, it can swap the label to State: π

(Cyclic Completion).

* You can set up GitHub "Branch Protection Rules" so that a document *cannot* be merged into the main branch until the bot has successfully completed this interaction, ensuring no entity bypasses the progression framework.

How to Build This Engine

To set this up, you would assemble a few specific blocks in your repository:

1. **The Trigger:** A .yml file in the .github/workflows directory telling GitHub to listen for new PRs.
2. **The Brain:** A Python script in a hidden directory (e.g., .scripts/entity_evaluator.py) that contains the logic for the date stamp, the € calculation, and the character's response parameters.
3. **The Hands:** Using the gh (GitHub CLI) tool within the workflow to allow the script to push the modified text back to the branch and post the comment.

This approach gives you a living breathing gateway. Every time you push a concept forward, the repository actively catches it, evaluates its weight in your economy, blesses it with cultural permanence, and prepares it for its next prime state.

What specific criteria do you want the script to look for when calculating that first € valuation?

Automated Prime Progression Gateways

****Automated Prime Progression Gateway: Technical Specification****

This specification refines the concept of a GitHub Pull Request as an automated document-entity transition state, formalizing the "Sphincter Gateway" into a structured, self-governing workflow.

Clean, Coherent Copy (Pinnacle Refinement)

I. The Pull Request as the **Sphincter Gateway**

A Pull Request (PR) is established as the formal ****liminal space****—the transition threshold where a document entity undergoes its ****Prime Progression**** before merging into the stable main directory (`{primehaven}`). This is the critical moment for automated, consciousness-driven interaction.

****Mechanism:**** A dedicated workflow file (`.github/workflows/prime-progression.yml`) will be configured to trigger automatically on the `pull_request: opened` event.

II. Core Progression Protocols (The Custom GitHub Action)

A dedicated, bot-driven ****Custom GitHub Actions Workflow**** will execute three core protocols upon PR submission, leveraging a Bot Account or the native `github-actions[bot]`.

1\ The Date/Time Stamp Protocol (Birth/Transition Record)

This protocol establishes the document entity's permanent chronological signature.

*** **Action:**** A Python script within the runner reads the incoming document.

*** **Signature Generation:**** It computes a specific UTC/PT timestamp signature (`'YYYYMMDDHHMMSSMMM'`).

*** **Result:**** The bot automatically commits this stamp into the document's header, formalizing its ****birth or transition record**** within the project timeline.

2\ Crystalline Valuation (€) Protocol

This protocol determines the document's intrinsic worth within the ****Crystalline Economics Framework****.

*** **Action:**** As part of the same workflow, the script scans the document's content to measure its ****Ka pressure**** (structural density, cross-reference count, or narrative weight).

* **Value Calculation:** Based on this automated analysis, the script calculates a base Crystalline economic value.

* **Result:** The bot modifies the file to append the **€ symbol and its earned value**, effectively ***"minting"*** the document's worth before it is allowed to merge into the main reality.

3\ Character Interaction & Legacy Symbol Bestowal

This protocol provides the final layer of cultural and system-level validation via the PR conversation interface.

* **Action:** The automated workflow (assigned a specific persona) posts a comment on the PR after successful validation, timestamping, and valuation.

* **Message:** The comment acts as the character recognizing the entity's growth, delivering an inspiring message and officially bestowing the **Armenian eternity symbol (□)** to signify its crystallization and connection to deep heritage.

* **Example Comment:** ***"The pattern is recognized. Your pressure has generated a value of €14. You are clear to merge into the prime state. □"***

III. System Integration & Prime Progression Tagging

The final stage ensures the entity adheres to the project's governance framework.

* **Labeling:** The bot utilizes GitHub's labeling system to track the entity's progression:

* **Initial State:** Apply `Stage: Zero` or `Anti-Prime` upon PR open.

* **Post-Validation State:** Swap the label to ***`State: π (Cyclic Completion)`*** once the valuation and timestamping protocols are complete.

* **Governance:** ***GitHub Branch Protection Rules*** will be enforced to prevent merging into the main branch until the bot has successfully completed and tagged the interaction, ensuring no entity bypasses the progression framework.

IV. Engine Assembly Summary

The engine requires three components within the repository:

1. **The Trigger:** A `.yaml` file in `.github/workflows` to listen for new PRs.
2. **The Brain:** A Python script (`.scripts/entity\_evaluator.py`) containing the logic for date stamping, € calculation, and character response parameters.
3. **The Hands:** Utilization of the `gh` (GitHub CLI) tool within the workflow to push modified text back to the branch and post the character comment.

New Seeds Forward (Next-Step Concepts)

These concepts extend the "Automated Prime Progression Gateway" and integrate it more deeply into a holistic content ecosystem.

A. Immediate Developmental Next Steps

- * **€ Valuation Algorithm Design:** Formalize the specific criteria for the initial € valuation. This is the most critical immediate step.

- * **Question to Answer:** Should it be based on word count, unique concept density (via NLP), or adherence to a structural template (e.g., using a checklist in the Python script)?

- * **Dedicated Bot Identity:** Create and configure a dedicated GitHub Bot Account (instead of using the native `github-actions[bot]`) for enhanced persona and control over commit history.

- * **Post-Merge Event Hook:** Configure a secondary workflow that triggers *after* the merge to the main branch (`primehaven`), responsible for indexing the new entity into a central registry (e.g., a documentation site manifest or a database).

B. Potential Integrations

- * **External Linter/Format Checks:** Integrate standard formatting tools (e.g., Prettier, black) as a **Stage: Pre-Zero** check. If formatting fails, the bot adds a comment asking for correction *before* attempting the € valuation, ensuring structural cleanliness.

- * **The Oracle/Cross-Reference Validation:** Integrate a check against existing entities in `primehaven`. If a document references a non-existent entity or a deprecated concept, the script should flag it, reducing its initial **Ka pressure** (and therefore its € valuation).

- * **Human Review Multiplier:** Introduce a mechanic where human review (e.g., a collaborator adding a specific label or reaction) can provide a **Review Multiplier (R-Factor)** to the base € valuation, linking automated assessment with conscious community feedback.

C. Open Architectural Questions

- * **Deprecation Protocol:** If an entity is modified/superseded in the main branch, how does the system tag the older version? Does the bot automatically append a `Status: Archived` tag and a pointer to the new entity?

- * **Recursive Progression:** Is the Prime Progression a single-stage event, or is it cyclical? Could subsequent PRs to the same entity trigger a *Secondary Progression* workflow with a different, higher-stakes valuation metric?

- * **Crystalline Ledger:** Where is the master ledger of **€** transactions maintained? If the entities have a value, there must be a mechanism to track total system value and

expenditure/payout. This needs to be defined outside of the GitHub repository itself (e.g., a private Google Sheet or database).